

PROJETO DE PROGRAMAÇÃO ORIENTADA A OBJETOS

Crônicas do Reino por dentro do backend

Guia completo para apresentar a arquitetura, os casos de uso, Builder, Strategy, Observer, SOLID, persistência, segurança e testes.

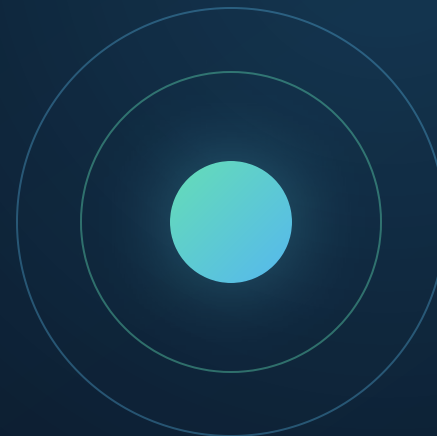
Python

FastAPI

SQLite

Clean Architecture

SOLID



Uma apresentação, dois integrantes, o mesmo domínio

Parte principal

Os slides 1–26 contam uma sequência: problema → arquitetura → regras → padrões → SOLID → infraestrutura → qualidade.

Tempo sugerido: 18 a 24 minutos.

Apêndice de defesa

Os slides 27–30 organizam a dupla, antecipam perguntas e deixam respostas curtas para a arguição.

Regra: dividir a fala, não dividir o conhecimento.

Frase de abertura: “Nosso backend separa regra de negócio, orquestração e detalhes técnicos para que o jogo possa evoluir sem transformar cada mudança em uma alteração no sistema inteiro.”

O que o backend realmente entrega

Identidade

Cadastro, login, logout, sessões e perfis: Jogador, Mestre e Administrador.

Personagens

Classe, raça, atributos, evolução, recuperação, habilidades e equipamentos.

Missões

Disponibilidade por nível, aceite, progresso, conclusão, cancelamento e recompensas.

Combate

Turnos, d100, agilidade, ataques, habilidades, energia, dano, fuga e recompensas.

Administração

Catálogos de classes, raças, habilidades, itens, missões e inimigos.

Rastreabilidade

Eventos de domínio alimentam um histórico persistido por personagem.

O frontend apenas consome essas capacidades. **A decisão sobre o que é permitido fica no backend**, não na tela.

Stack escolhida e motivo de cada peça

PEÇA	PAPEL	POR QUE FOI USADA
Python	Implementação do backend	Leitura simples, orientação a objetos e biblioteca padrão suficiente para boa parte do projeto.
FastAPI + Pydantic	HTTP, rotas e validação de entrada	Tipagem clara, injeção de dependências e documentação Swagger automática em /docs .
sqlite3	Persistência relacional	Banco local, transacional e adequado ao porte acadêmico; sem custo de um ORM.
dataclasses	Entidades e eventos	Objetos explícitos com pouco código repetitivo.
unittest + httpx	Validação automatizada	Testes unitários, integrados e HTTP com banco em memória.

Escolha consciente: não usar ORM simplifica este escopo. A existência de ports permite trocar SQLite por outra implementação depois.

A ideia central: a regra manda; o framework obedece

Entidades e regras de domínio

Casos de uso / aplicação

Adaptadores de interface

FastAPI, SQLite, segurança e I/O

Regra de dependência

As dependências de código apontam para as camadas mais internas.

- Entidades não conhecem HTTP nem SQLite.
- Casos de uso não importam o repositório concreto.
- A infraestrutura implementa contratos definidos pela aplicação.
- O container faz a ligação entre abstração e implementação.

Benefício: trocar banco, dado aleatório ou forma de entrada afeta a borda — não a regra central do RPG.

Como as camadas aparecem nas pastas

app/entities

Objetos do negócio: Character, Mission, Item, Enemy, Combat, User etc. Character e MissionProgress concentram regras de estado.

app/domain

Enums, erros, eventos e padrões: CharacterBuilder, estratégias de ataque e DomainEvent.

app/use_case

UC01 Criar Personagem, UC02 Aceitar Missão e UC03 Realizar Combate, todos explícitos.

app/application

Ports, autorização e serviços auxiliares de autenticação, inventário, missão, administração e histórico.

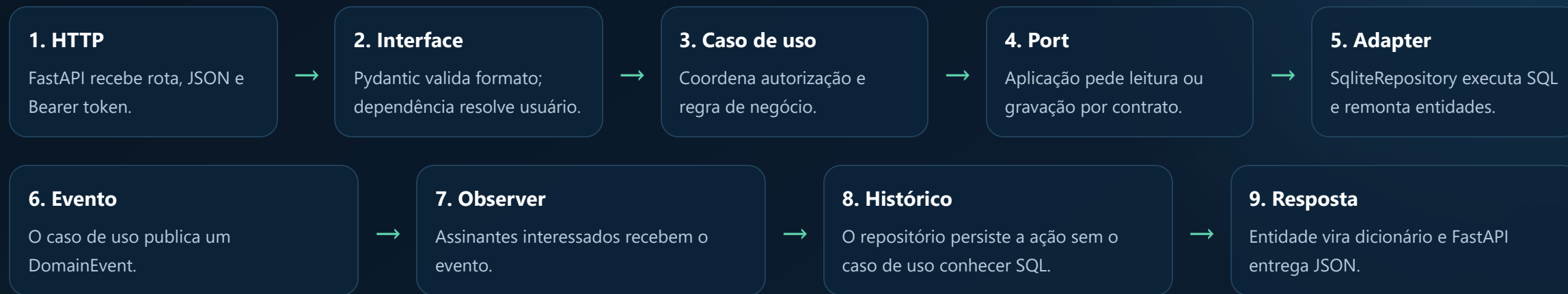
app/infrastructure

SQLite, repositório, script, token, UUID, d100, eventos em memória, migração e seed.

app/interfaces

API FastAPI: recebe JSON, autentica, chama aplicação e transforma erros em respostas HTTP.

O caminho de uma requisição



Exemplo: `POST /api/characters` → `CreateCharacterUseCase.execute` → `CharacterRepository` → `SQLite` → evento `CHARACTER_CREATED`.

Container: o único lugar que conhece todas as peças

```
def create_container(config, dice_roller=None):  
    database = Database(config.database_path)  
    repository = SqliteRepository(database)  
    events = InMemoryEventPublisher()  
    events.subscribe("*", repository.add_history)  
  
    use_case = PerformCombatUseCase(  
        repository, repository, repository,  
        events, authorization, id_generator,  
        dice_roller or D100DiceRoller(),  
    )
```

Composition Root

`app/container.py` instancia os objetos concretos e os injeta nos casos de uso.

- Centraliza a configuração.
- Evita `new SqliteRepository()` dentro da regra.
- Permite injetar um dado falso nos testes.
- Materializa Clean Architecture e DIP.

Frase-chave: "O caso de uso sabe o que precisa; o container decide quem fornece."

Entidade não é apenas uma tabela

Catálogo e identidade

`User`, `CharacterClass`, `Race`, `Skill`, `Item`, `Mission`, `Enemy` e `Combat` representam conceitos do jogo e convertem seus dados para o contrato JSON.

Entidades com comportamento rico

`Character` recebe dano, gasta energia, evolui, distribui atributos e se recupera. `MissionProgress` controla progresso, conclusão e cancelamento.

```
character.receive_damage(damage)    # vida nunca fica negativa
character.spend_energy(cost)        # impede gasto sem energia
levels = character.gain_experience(xp)
progress.complete()                # exige meta cumprida
```

Nem toda entidade precisa ter muitas regras. Dados de catálogo são mais simples; estados críticos ficam protegidos pelos objetos que realmente os governam.

Casos de uso explícitos × serviços auxiliares

Casos de uso exigidos

- `CreateCharacterUseCase.execute` — UC01.
- `AcceptMissionUseCase.execute` — UC02.
- `PerformCombatUseCase` — UC03, com início, turno e listagem.

Cada classe descreve uma intenção do usuário e coordena entidades, ports, autorização e eventos.

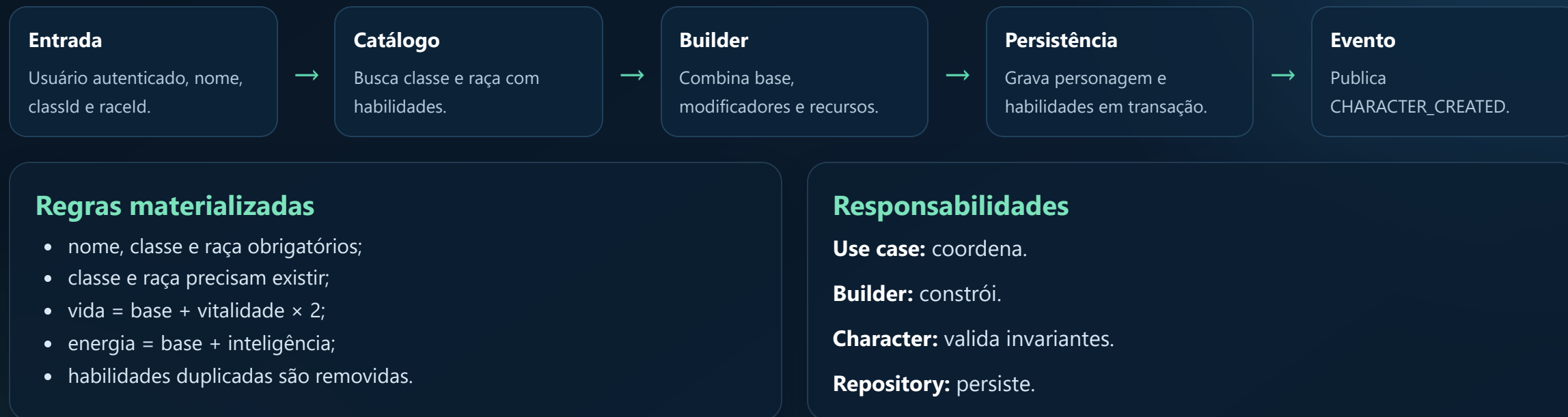
Serviços de apoio

- `AuthService`
- `CharacterService` e `MissionService`
- `InventoryService`, `AdminService` e `HistoryService`

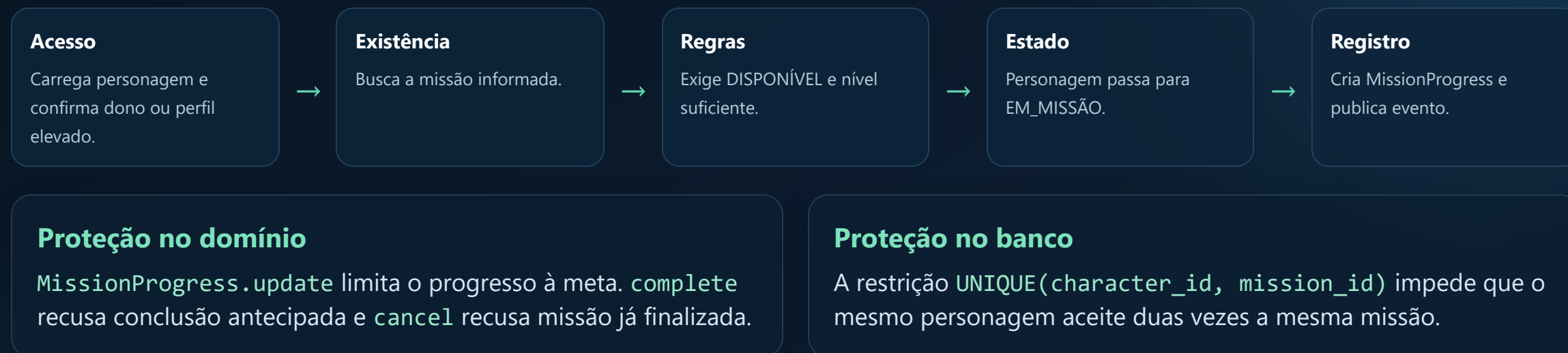
Agrupam operações relacionadas que apoiam o produto, sem esconder os três UCs centrais do documento.

Diferença: entidade decide como seu estado muda; caso de uso decide a sequência da ação; repositório decide como persistir.

Criar personagem: da escolha ao objeto válido



Aceitar missão: autorização antes da mudança de estado



A mesma regra é defendida em níveis diferentes: aplicação comunica bem o erro; banco garante integridade mesmo diante de concorrência.

Realizar combate: o fluxo mais completo

Início

1. Autoriza acesso ao personagem.
2. Exige estado ATIVO ou EM_MISSÃO.
3. Busca inimigo e cria combate.
4. Persiste estado EM_COMBATE.

Turno

1. Seleciona Strategy de ataque.
2. Valida habilidade, nível e energia.
3. Rola d100 e compara agilidades.
4. Aplica dano, contra-ataque e resultado.

Acerto e dano

$\text{acerta se } d100 + \text{AGI atacante} - \text{AGI defensor} > 70$

Dano considera atributos, defesa e bônus de equipamento, com mínimo de 1 quando o golpe acerta.

Encerramento

Vitória entrega XP, moedas e possível item. Derrota muda o estado do personagem. Fuga encerra sem recompensa. Tudo fica no histórico.

Três problemas diferentes, três padrões adequados

01

Builder

Categoria: criacional.

Monta um personagem complexo em etapas legíveis e entrega um objeto válido.

02

Strategy

Categoria: comportamental.

Encapsula algoritmos de ataque diferentes atrás do mesmo contrato.

03

Observer

Categoria: comportamental.

Notifica interessados sobre eventos sem acoplar quem produz a quem reage.

Importante: Repository e Dependency Injection também aparecem como soluções arquiteturais, mas os três padrões de projeto escolhidos e defendidos são Builder, Strategy e Observer.

Por que usamos e como implementamos

```
character = (  
    CharacterBuilder()  
        .owned_by(user["id"])  
        .named(name)  
        .from_class(character_class)  
        .from_race(race)  
        .build()  
)
```

Problema resolvido

Um personagem nasce de dados do jogador, da classe e da raça. Um construtor enorme misturaria busca, combinação e validação.

Solução

`CharacterBuilder` mantém dados temporários, expõe etapas encadeáveis e só cria `Character` em `build()`.

Benefícios

- leitura próxima do caso de uso;
- ordem de montagem explícita;
- fórmulas centralizadas;
- objeto final validado.

Não é só “encadear métodos”: Builder separa o processo de construção da representação final do personagem.

Trocar o algoritmo sem trocar o fluxo de combate

```
class AttackStrategy(ABC):
    @abstractmethod
    def execute(self, attacker, defender): ...

class BasicAttackStrategy(AttackStrategy): ...
class SkillAttackStrategy(AttackStrategy): ...

result = select_attack_strategy(action, skill) \
    .execute(character, enemy)
```

O que varia?

A fórmula, o atributo de escala, o custo de energia e as validações.

O que permanece?

O turno continua rolando d100, aplicando dano, persistindo e publicando eventos.

Por que é melhor?

Evita um método de combate cheio de condicionais sobre cada tipo de ataque.

Limite atual: a função seletora ainda conhece as ações. Para muitos tipos futuros, um registro de estratégias poderia tornar a extensão totalmente configurável.

Histórico sem SQL dentro dos casos de uso

Produtor

Use case cria DomainEvent.

**Publisher**

InMemoryEventPublisher publica.

**Assinantes**

Handlers do tipo ou curinga "*".

**Efeito**

repository.add_history persiste.

```
events.subscribe("*", repository.add_history)

self.events.publish(DomainEvent(
    "MISSION_COMPLETED", user["id"], character.id,
    "Missão concluída.", {"experience": xp}
))
```

Ganho imediato

O caso de uso só anuncia um fato. Ele não sabe qual tabela receberá o histórico.

Evolução possível

Notificação, conquista e auditoria podem assinar o mesmo evento sem alterar o produtor.

Os padrões não ficam isolados: eles sustentam os fluxos

MOMENTO	PADRÃO ATUANTE	RESULTADO
Criação do personagem	Builder	Combina classe + raça + jogador e cria uma entidade coerente.
Execução do turno	Strategy	Escolhe o cálculo correto sem duplicar o ciclo do combate.
Após ações importantes	Observer	Registra criação, missão, combate, nível, atributo e item no histórico.
Em todos os momentos	Ports + DI	Casos de uso trabalham com contratos, e o container fornece implementações.

“Builder organiza a criação; Strategy organiza a variação; Observer organiza a reação.”

Cinco princípios para controlar acoplamento e mudança

S**Single Responsibility**

um motivo principal para mudar.

O**Open/Closed**

estender sem reescrever o núcleo.

L**Liskov Substitution**

subtipos preservam contratos.

I**Interface Segregation**

contratos pequenos e focados.

D**Dependency Inversion**

regra depende de abstrações.

SOLID não é uma coleção de pastas. É uma forma de decidir **quem conhece quem, quem muda por qual motivo e como substituir comportamentos.**

Responsabilidade única e abertura para extensão

S — Single Responsibility

Definição: uma classe deve ter um motivo principal para mudar.

- `AuthService`: autenticação e sessão.
- `AuthorizationPolicy`: decisões de permissão.
- `Character`: estado e regras do personagem.
- `Database`: conexão, transação e schema.
- `create_app`: borda HTTP.

O — Open/Closed

Definição: aberto para extensão, fechado para modificação desnecessária.

- Nova Strategy pode implementar `AttackStrategy`.
- Novo Observer pode assinar eventos.
- Novo banco pode implementar os ports.
- Novo dado pode implementar `DiceRoller`.

Análise honesta: `SqliteRepository` é amplo por simplicidade acadêmica. Ele tem uma responsabilidade arquitetural — persistência SQLite — mas poderia ser dividido por agregado em uma evolução maior.

Substituição segura e interfaces focadas

L — Liskov Substitution

Definição: uma implementação concreta deve poder substituir sua abstração sem quebrar o cliente.

`D100DiceRoller` e `FixedDiceRoller` oferecem `roll_d100()`. O combate funciona com ambos: aleatório em produção, previsível em teste.

`SqliteRepository` respeita os métodos definidos pelos repository ports.

I — Interface Segregation

Definição: clientes não devem depender de métodos que não usam.

Em vez de uma interface gigante, há `UserRepository`, `CharacterRepository`, `MissionRepository`, `CombatRepository`, `DiceRoller`, `EventPublisher` etc.

Um caso de uso declara somente os contratos necessários em seu construtor.

Prova prática: o teste troca o dado real por um dado controlado sem alterar uma linha do `PerformCombatUseCase`.

Dependency Inversion: o centro não depende da borda

```
class PerformCombatUseCase:
    def __init__(
        self,
        combats: CombatRepository,
        characters: CharacterRepository,
        catalog: CatalogRepository,
        events: EventPublisher,
        dice_roller: DiceRoller,
    ): ...
```

O princípio

Módulos de alto nível e detalhes de baixo nível dependem de abstrações.

No projeto

- Use cases dependem dos ABCs em `ports.py`.
- SQLite, scrypt e d100 implementam esses contratos.
- O container injeta os objetos concretos.

Resultado

Regra testável, menor acoplamento e infraestrutura substituível.

DIP é a ponte entre SOLID e Clean Architecture neste backend.

SQLite sem deixar SQL vaziar para o negócio

Database

Abre conexão, ativa chaves estrangeiras, WAL e lock; oferece execução, consulta e transação com rollback.

Schema

Tabelas para usuários, sessões, catálogos, personagens, inventário, missões, combates, turnos e histórico.

Repository

Executa SQL, traduz linhas em entidades e converte conflitos do banco em erros da aplicação.

Integridade

- foreign keys;
- campos obrigatórios e CHECK;
- e-mail e nomes únicos;
- missão única por personagem;
- operações críticas em transação.

Mapeamento

O banco usa `snake_case`; a API entrega `camelCase`. O repositório monta `User`, `Mission`, `Enemy`, `Combat` etc. e chama `to_dict()`.

Autenticação, autorização e respostas de erro

Senha

`hashlib.scrypt` com salt aleatório de 16 bytes.
Comparação em tempo constante com `compare_digest`.

Sessão

Token aleatório de 32 bytes; só o SHA-256 é persistido. Sessão expira, por padrão, após 12 horas.

Permissão

`AuthorizationPolicy` distingue Jogador, Mestre e Administrador e protege propriedade do personagem.

ERRO DE DOMÍNIO	HTTP	EXEMPLO
AuthenticationError	401	Token ausente, inválido ou expirado.
AuthorizationError	403	Jogador tenta administrar catálogo.
NotFoundError	404	Missão ou personagem inexistente.
ConflictError	409	Energia insuficiente ou estado incompatível.
ValidationError	422	Campo inválido ou regra de entrada violada.

Como sabemos que o backend continua funcionando?

16 testes automatizados

- **Domínio:** Builder, Strategy, nível, atributos, derrota, recuperação e missão.
- **Estrutura:** entidades e casos de uso explícitos.
- **Integração:** conta → personagem → missão → combate → histórico.
- **API:** autenticação, permissões, catálogos, inventário, Swagger e health.

Testabilidade gerada pela arquitetura

- SQLite `:memory:` isola cada teste.
- `FixedDiceRoller` elimina aleatoriedade.
- `httpx.ASGITransport` exercita HTTP sem servidor externo.
- Seed é testado como idempotente.

```
python -m unittest discover -v  
python -m compileall -q app test
```

Resultado validado: 16 testes aprovados, compilação Python e verificação do JavaScript sem erros.

O que esta arquitetura nos entrega

Clareza

Pastas e classes revelam intenção: entidade, use case, port, adapter e interface.

Proteção

Regras centrais não ficam espalhadas entre rota, tela e SQL.

Testabilidade

Dependências substituíveis tornam fluxos complexos determinísticos.

Evolução

Novas estratégias, observadores ou adapters podem ser adicionados com impacto localizado.

Simplicidade

SQLite e biblioteca padrão atendem o porte do projeto sem infraestrutura excessiva.

Rastreabilidade

Eventos registram o que aconteceu sem acoplar a regra ao histórico.

“Não usamos padrões por enfeite: cada padrão responde a uma variação real do RPG.”

Divisão de fala sem divisão de conhecimento

Integrante 1 — fala principal

Visão geral, stack, Clean Architecture, camadas, fluxo HTTP, entidades, UC01 e Builder.

Também precisa saber responder: Strategy, Observer, SOLID, segurança, banco e testes.

Integrante 2 — fala principal

UC02, UC03, Strategy, Observer, SOLID, persistência, segurança, testes e conclusão.

Também precisa saber responder: Clean Architecture, entidades, UC01, Builder, ports e container.

Ensaio 1 Cada um apresenta sua parte e o outro interrompe com perguntas.

Ensaio 2 Troquem completamente as partes. Se ambos conseguem explicar tudo, a dupla está pronta.

Ensaio 3 Apresentação normal, com transições curtas e respostas alternadas.

Transição sugerida: “Agora que vimos como as camadas se comunicam, meu colega vai mostrar como os padrões tornam esses fluxos flexíveis.”

Perguntas prováveis do professor — arquitetura

“Por que o use case não acessa SQLite?”

Porque a regra depende de repository ports. Isso preserva DIP e a regra de dependência da Clean Architecture.

“O que muda se trocar o banco?”

Cria-se outro adapter que implemente os mesmos ports e altera-se o container. A intenção dos casos de uso permanece.

“Entidade é igual a tabela?”

Não. Entidade representa conceito e comportamento do negócio; tabela é uma forma de persistir seu estado.

“Por que use_case e application?”

`use_case` deixa explícitos os três UCs exigidos. `application` contém contratos e operações auxiliares do sistema.

“Por que um repositório concreto grande?”

É uma simplificação do porte acadêmico. Os consumidores continuam usando interfaces pequenas; separar adapters por agregado é evolução possível.

“FastAPI faz parte do domínio?”

Não. É detalhe externo na camada de interface; recebe HTTP e delega.

Perguntas prováveis — padrões e SOLID

“Por que Builder?”

Porque Character nasce da combinação de várias fontes e fórmulas. O padrão separa montagem da entidade final e deixa o UC01 legível.

“Strategy é só um if?”

O seletor tem uma decisão pequena; os algoritmos completos estão em objetos substituíveis sob o mesmo contrato. É essa encapsulação que caracteriza Strategy.

“Observer é assíncrono?”

Neste projeto, não: é síncrono e em memória, adequado ao escopo. O desacoplamento entre produtor e assinante continua existindo.

“Qual exemplo mais claro de LSP?”

O dado real e o dado fixo podem ocupar o papel de DiceRoller sem mudar o combate.

“Qual princípio sustenta a Clean Architecture?”

Principalmente DIP: o núcleo depende de abstrações; detalhes externos implementam essas abstrações.

“O OCP está perfeito?”

Ele é bem atendido por strategies e observers. Para muitos ataques, um registro de strategies removeria a necessidade de alterar o seletor.

Se os dois conseguem explicar isto, estão prontos

Arquitetura

- ✓ Caminho rota → use case → port → repository.
- ✓ Função de cada pasta.
- ✓ Regra de dependência e papel do container.
- ✓ Diferença entre entidade, caso de uso e tabela.

Decisões

- ✓ Problema, implementação e ganho de cada padrão.
- ✓ Um exemplo real para cada letra de SOLID.
- ✓ Fluxos UC01, UC02 e UC03.
- ✓ Segurança, persistência e evidências dos testes.

Fechamento sugerido: "Nosso backend traduz as regras do RPG em objetos e casos de uso testáveis, deixando FastAPI e SQLite como detalhes substituíveis nas bordas."

Perguntas?